CURSO DE ENGENHARIA DE SOFTWARE
DISCIPLINA: TESTES DE SOFTWARE

Mirelly Pego Cordeiro de Alvarenga

TESTE DE MUTAÇÃO COM STRYKERJS: AVALIAÇÃO E FORTALECIMENTO DA SUÍTE DE TESTES DA “CALCULADORA MUTANTE”

Belo Horizonte
2026

1. INTRODUÇÃO

Este relatório documenta a aplicação de teste de mutação sobre o projeto "Calculadora Mutante", uma biblioteca JavaScript com 50 funções matemáticas e uma suíte inicial de 50 testes em Jest. Utilizou-se a ferramenta StrykerJS, que gera versões alteradas do código, os mutantes, e verifica se a suíte é capaz de detectá-las. O objetivo foi medir a real eficácia da suíte e fortalecê-la até superar 98% de pontuação de mutação. E o repositório se encontra nesse link: MirellyAlvarenga/operacoes-mutante.

2. ANÁLISE INICIAL

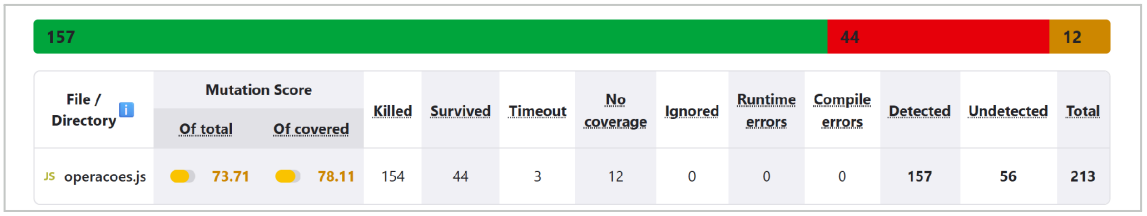
Com a suíte original, todos os 50 testes passaram. O relatório de cobertura e a primeira execução do Stryker produziram os valores da Tabela 1.

Tabela 1 – Cobertura de código e pontuação de mutação iniciais

Métrica	**Valor inicial**
Cobertura de linhas	98,64%
Cobertura de funções	100,00%
Cobertura de instruções (statements)	85,41%
Cobertura de ramos (branches)	58,82%
Pontuação de mutação (total)	**73,71%**

Fonte: Jest (relatório de cobertura) e StrykerJS.

Figura 1 – Resumo da primeira execução do StrykerJS



Fonte: Relatório HTML do StrykerJS (reports/mutation/mutation.html).

A discrepância é evidente: quase todas as linhas (98,64%) e todas as funções (100%) eram executadas pelos testes, mas apenas 73,71% dos mutantes foram detectados, cerca de um em cada quatro defeitos artificiais passava despercebido. Isso ocorre porque a cobertura mede apenas se o código foi executado, e não se o comportamento foi verificado.

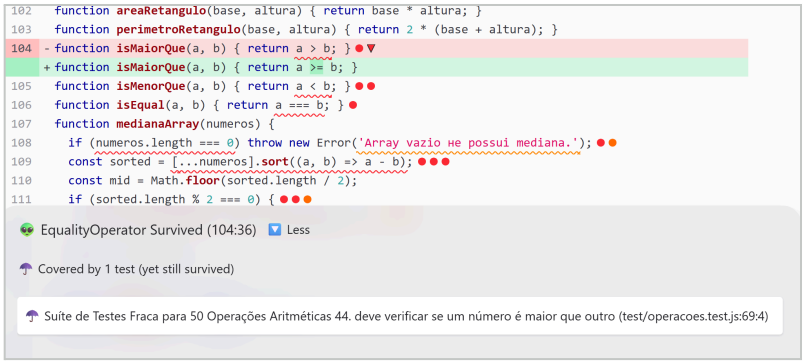
Cada teste original exercitava um único "caminho feliz", com valores que não distinguem a operação correta de uma errada, e nenhum verificava fronteiras, retornos false ou erros. A baixa cobertura de ramos (58,82%) já era um indício dessa fragilidade, que a cobertura de linhas escondia.

3. ANÁLISE DE MUTANTES CRÍTICOS

3.1. Mutante 1: operador relacional em isMaiorQue (linha 104)

O Stryker substituiu a > b por a >= b. O único teste existente, expect(isMaiorQue(10, 5)).toBe(true), produz true nas duas versões, pois 10 é tanto maior quanto "maior ou igual" a 5. O teste nunca avaliou a fronteira a === b, exatamente onde os dois operadores divergem, nem um caso em que o resultado deveria ser false, por isso também sobreviveu o mutante que troca toda a expressão por return true. O mesmo padrão ocorreu em isMenorQue, isEqual, fatorial e isPrimo.

Figura 2 – Mutante EqualityOperator sobrevivente em isMaiorQue

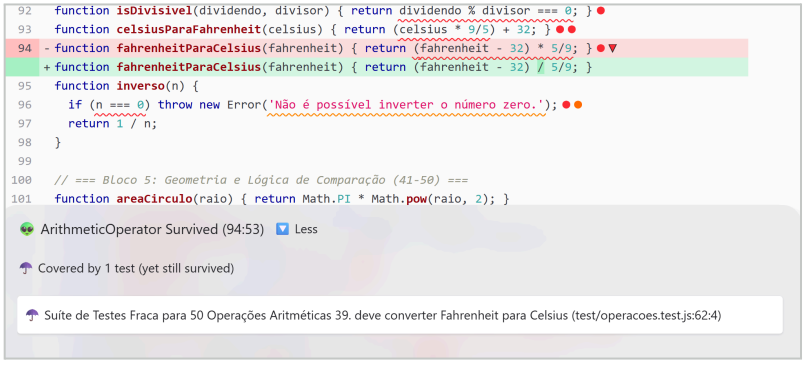


Fonte: Relatório HTML do StrykerJS, primeira execução.

3.2. Mutante 2: operador aritmético em fahrenheitParaCelsius

A mutação trocou (fahrenheit - 32) * 5/9 por (fahrenheit - 32) / 5/9. O teste original usava a entrada 32 °F, para a qual (32 - 32) vale zero; como zero multiplicado ou dividido por qualquer fator continua zero, o resultado esperado (0) é obtido mesmo com a fórmula errada. A entrada escolhida "neutraliza" justamente a operação que deveria ser testada. O mesmo ocorreu em celsiusParaFahrenheit(0): com 0 °C, o fator 9/5 é multiplicado por zero e qualquer alteração nele sobrevive.

Figura 3 – Mutante ArithmeticOperator sobrevivente em fahrenheitParaCelsius

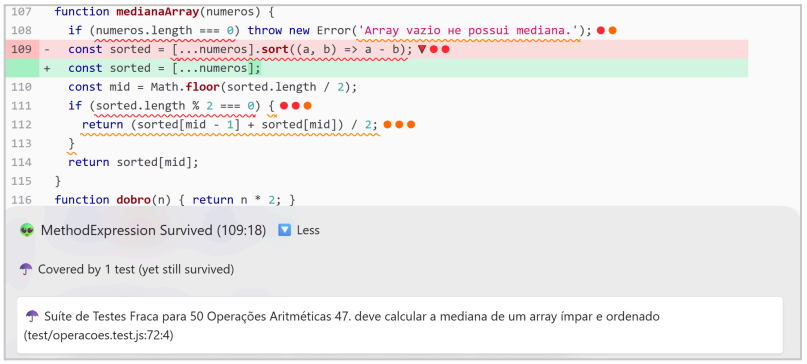


Fonte: Relatório HTML do StrykerJS, primeira execução.

3.3. Mutante 3: remoção da ordenação em medianaArray (linha 109)

O Stryker removeu a chamada .sort((a, b) => a - b), deixando o array na ordem original. O teste usava [1, 2, 3, 4, 5], que já está ordenado; assim, a ordenação não tinha efeito observável e sua remoção passou despercebida (o mesmo vale para os mutantes que alteram o comparador). Além disso, por usar apenas um array de tamanho ímpar, o ramo de tamanho par nunca foi executado, o que gerou mutantes sem cobertura.

Figura 4 – Mutante MethodExpression sobrevivente em medianaArray



Fonte: Relatório HTML do StrykerJS, primeira execução.

4. SOLUÇÃO IMPLEMENTADA

Foram adicionados 27 novos testes em um novo bloco describe do arquivo test/operacoes.test.js, elevando a suíte de 50 para 77 testes, todos aprovados no código original. Para os mutantes analisados, as principais asserções adicionadas foram:

```
expect(isMaiorQue(5, 5)).toBe(false);          // fronteira a === b
expect(isMaiorQue(5, 10)).toBe(false);         // caso negativo
expect(fahrenheitParaCelsius(212)).toBe(100);  // entrada não neutra
expect(medianaArray([5, 1, 3])).toBe(3);       // array desordenado
expect(medianaArray([3, 10, 1, 2, 7])).toBe(3);
expect(medianaArray([4, 1, 3, 2])).toBe(2.5);  // ramo par
expect(medianaArray([10, 20])).toBe(15);
```

Esses testes são eficazes porque cada entrada foi escolhida para que o código original e o código mutado produzam resultados diferentes:

- **isMaiorQue(5, 5):** o original retorna false, enquanto a >= b e return true retornam true, e o teste falha. O caso (5, 10) garante que o retorno false também seja verificado fora da fronteira.
- **fahrenheitParaCelsius(212):** o original calcula 180 × 5/9 = 100, enquanto o mutante calcula 180 / 5 / 9 = 4 (e a variante * 5 * 9, 8100). Com uma entrada diferente de 32, o fator de conversão passa a influenciar o resultado.

- **medianaArray([5, 1, 3]):** com ordenação, o elemento central é 3; sem ordenação, seria 1. Arrays desordenados matam a remoção do sort e as alterações do comparador. Os arrays pares [4, 1, 3, 2] → 2,5 e [10, 20] → 15 exercitam o ramo par e matam mutantes como * 2 no lugar de / 2 (10) ou mid + 1 no lugar de mid - 1 (3,5).

Os demais testes seguiram o mesmo raciocínio: erros verificados com a mensagem exata via toThrow('mensagem'), o que também mata os mutantes que esvaziam a string; fronteiras como raizQuadrada(0), fatorial(0) e isPrimo(1); retornos false em isPar, isImpar, isDivisivel, isPrimo(4) e isEqual; e valores fora do intervalo em clamp. Restaram cinco mutantes equivalentes, que não alteram o comportamento e não podem ser mortos por nenhum teste: quatro na condição n === 0 || n === 1 de fatorial (o laço já retorna 1 para esses valores) e um em if (numeros.length === 0) return 1 de produtoArray (o reduce com valor inicial 1 já retorna 1). Eles foram marcados com // Stryker disable next-line, restrito aos mutadores envolvidos e com justificativa no código.

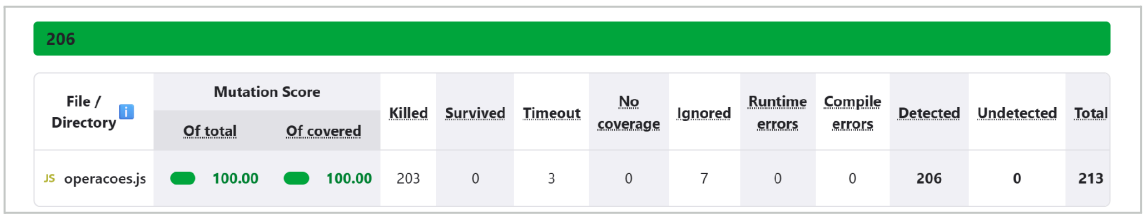
5. RESULTADOS FINAIS

Tabela 2 – Comparativo entre a execução inicial e a final

Métrica	**Inicial**	**Final**
Quantidade de testes	50	77
Cobertura de linhas	98,64%	100,00%
Cobertura de ramos (branches)	58,82%	97,05%
Mutantes detectados (mortos + timeout)	157	206
Mutantes sobreviventes	44	0
Mutantes sem cobertura	12	0
Mutantes ignorados (equivalentes)	0	7
Pontuação de mutação	**73,71%**	**100,00%**

Fonte: Jest e StrykerJS.

Figura 5 – Resumo da execução final do StrykerJS



Fonte: Relatório HTML do StrykerJS, execução final.

A pontuação de mutação passou de 73,71% para 100%, superando a meta de 98%, sem nenhum mutante sobrevivente ou sem cobertura. Os 7 ignorados são os 5

equivalentes e 2 variações if (true) nas mesmas linhas, também desativadas pelo filtro de mutador. Sem ignorar nenhum, a pontuação seria 97,65% (208 de 213).

6. CONCLUSÃO

O trabalho mostrou que uma cobertura de código alta não garante uma suíte de testes eficaz: com 98,64% das linhas cobertas, a suíte original deixava passar 56 dos 213 mutantes. A cobertura responde apenas “este código foi executado?”, enquanto o teste de mutação responde “se este código estivesse errado, algum teste perceberia?”. Cada sobrevivente apontou uma lacuna concreta, fronteiras não testadas, entradas que neutralizam a operação, falta de casos negativos e de verificação de erros, orientando testes mais precisos. Apesar do custo computacional maior e da necessidade de julgamento crítico, como no reconhecimento de mutantes equivalentes, o teste de mutação é uma ferramenta valiosa para avaliar a qualidade dos testes e garantir que eles realmente protejam o software contra regressões.